CO3 AT2-EXPERIMENTS

Name: Shaik Azeem

Reg. No: 192472307

Course Code: CSA0612

Course Name: DESIGN AND ANALYSIS OF ALGORITHM

Q1 **Online Advertisement Ranking (Quick Sort) An advertisement platform ranks ads based on relevance and bid value** 100 marks

Analyze how Quick Sort can be applied for efficient ranking. Identify issues such as pivot selection and data distribution. Apply divide and conquer techniques. Analyze performance and propose optimizations with justification.

Your Code

```
ads =[
    ("Ad A",130),
    ("Ad B",150),
    ("Ad C",125),
    ("Ad D",150),
    ("Ad E",90),
    ("Ad F",145)
]

def quick_sort(arr):
    if len(arr)<=1:
        return arr
    pivot= arr[len(arr)//2][1]

    left= [ad for ad in arr if ad[1]>pivot]
    middle= [ad for ad in arr if ad[1]==pivot]
    right= [ad for ad in arr if ad[1]<pivot]

    return quick_sort(left) + middle + quick_sort(right)
ranked_ads =quick_sort(ads)

print("Advertisement Ranking: ")
for rank,(name, score) in enumerate(ranked_ads,1):
    print(f"{rank}. {name} - Score: {score}")
```

Input

```
stdin input
```

Output

```
Advertisement Ranking:
1. Ad B - Score: 150
2. Ad D - Score: 150
3. Ad F - Score: 145
4. Ad A - Score: 130
5. Ad C - Score: 125
```

Objective:

The objective is to rank online advertisements based on their relevance and bid value. Advertisements with higher scores should appear at the top so that the most valuable and relevant ads are displayed first.

Problem Description:

An online advertising platform may receive a large number of advertisements. Each advertisement is assigned a ranking score, which can depend on factors such as:

- Bid amount
- Relevance to the user
- Advertisement quality
- User engagement

The advertisements need to be arranged in descending order of score.

Why Quick Sort?

Quick Sort is suitable for advertisement ranking because it uses the divide-and-conquer technique.

The main idea is:

1. Select a pivot advertisement.
2. Divide the remaining advertisements into groups based on the pivot score.
3. Recursively sort the divided groups.
4. Combine the groups to obtain the final ranking.

For the given example, the advertisements have scores such as:

Advertisement	Score
Ad A	130
Ad B	150
Ad C	125
Ad D	150
Ad E	90
Ad F	145

After sorting, the ranking is:

1. Ad B – 150
2. Ad D – 150
3. Ad F – 145
4. Ad A – 130
5. Ad C – 125
6. Ad E – 90

Divide and Conquer Approach:

Quick Sort divides the advertisements into three groups relative to a selected pivot:

- Higher score than pivot
- Equal score to pivot
- Lower score than pivot

Each group is then processed recursively until the advertisements are completely ordered.

Pivot Selection:

Pivot selection has an important effect on Quick Sort's performance.

- Good pivot: Divides the data into reasonably balanced groups.
- Poor pivot: Creates highly unbalanced groups.
- A middle element, random element, or median-of-three strategy can be used to improve performance.

Data Distribution Issues:

The performance of Quick Sort depends on how the advertisement scores are distributed.

- Randomly distributed scores: Usually gives good performance.
- Already sorted data: Can cause poor performance with an unsuitable pivot.
- Many duplicate scores: Can create unnecessary partitions if not handled properly.
- Highly unbalanced data: Increases the number of recursive operations.

Performance Analysis:

Case	Time Complexity
Best Case	O(n log n)
Average Case	O(n log n)
Worst Case	$O(n^2)$

The average performance is efficient for ranking a large number of advertisements.

Optimizations:

The following techniques can improve Quick Sort:

- Use random pivot selection to reduce the chance of worst-case behavior.
- Use median-of-three pivot selection for better partitioning.
- Handle duplicate scores efficiently.
- Use insertion sort for very small partitions.
- Avoid excessively deep recursion.

Conclusion:

Quick Sort provides an efficient divide-and-conquer approach for online advertisement ranking. It can quickly arrange advertisements according to their ranking scores. Proper pivot selection and handling of duplicate or unevenly distributed data are important for maintaining good performance, especially when the number of advertisements is large.